

コンテキストエンジニアリング入門

肥大化する情報に立ち向かう

イースト株式会社開発本部第四事業部

高瀬拓史

すべての始まり：LLMは記憶を持たない

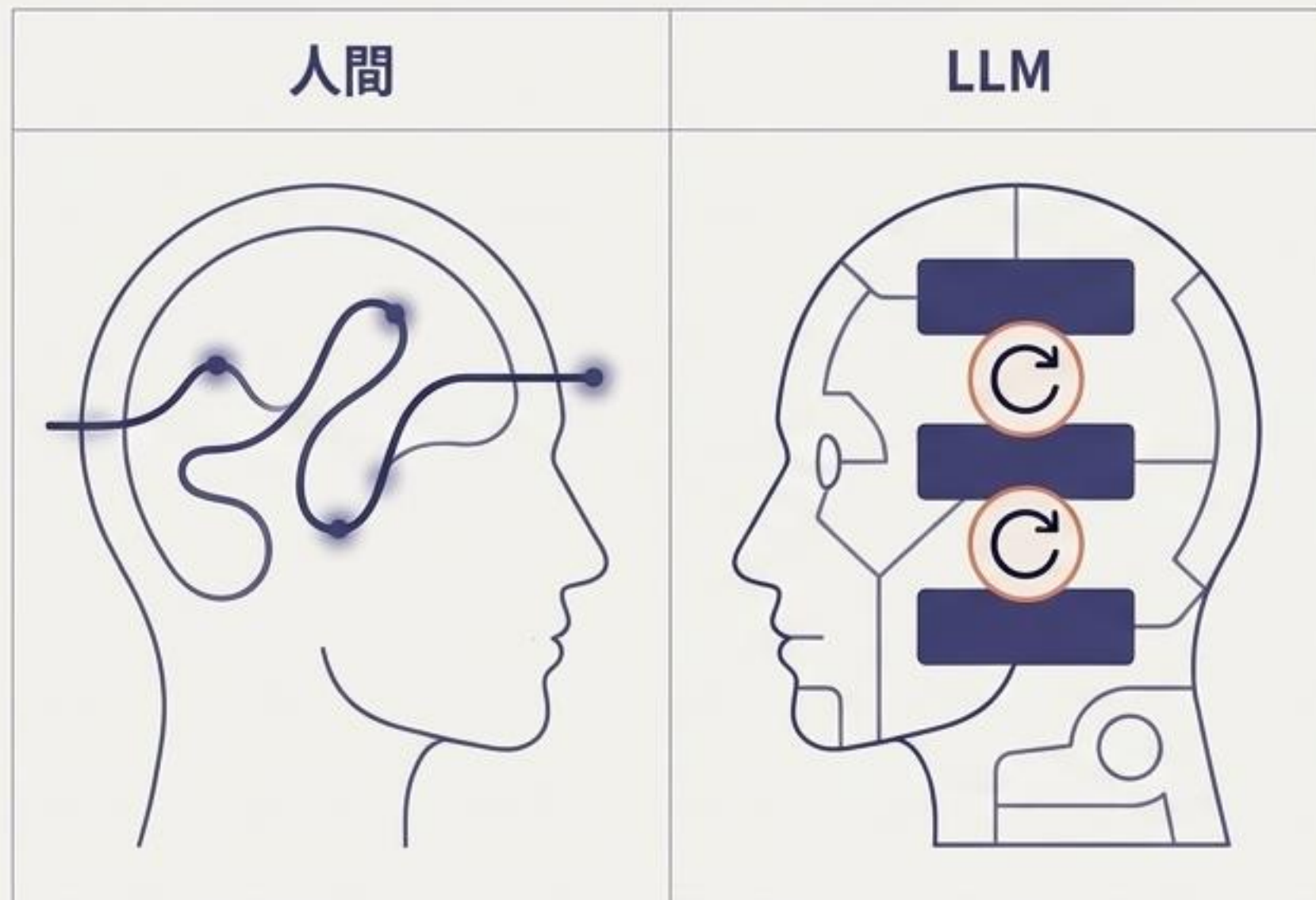
キーコンセプト

LLMは本質的にステートレス（状態を持たない）。

人間との違いのアナロジー

人間は「前回の続き」を覚えている。

LLMは「毎回、最初のページから読み直す」必要がある、記憶力のないアシスタントのようなもの。

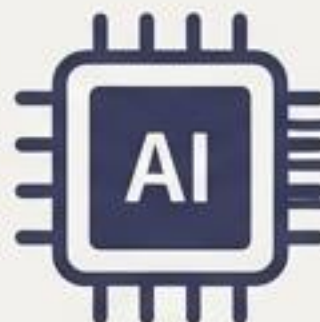


毎回の対話で、
AIは全ての情報を
ゼロから読み直す
必要がある。

AIはどのようにメッセージを処理するのか

処理フロー

1. メッセージ送信
2. システムプロンプトを読み込む
3. 過去の会話履歴を読み込む
4. 新しいメッセージを読み込む
5. 全てを考慮して応答を生成



システムプロンプト

AIの基本的な振る舞いを決める指示
(例: 「あなたは親切なアシスタントです」)



会話履歴

これまでのやり取りの記録。ステートレスなので毎回送信が必要。

新しいメッセージ

今回の質問や指示。

コンテキストウィンドウの限界：AIの作業机は有限である

コンテキスト

文脈・前後関係・背景情報。AIが判断に使う全ての情報（コード、規約、会話履歴など）。

コンテキストウィンドウ

AIが一度に読める情報量の上限。



会話が長くなる



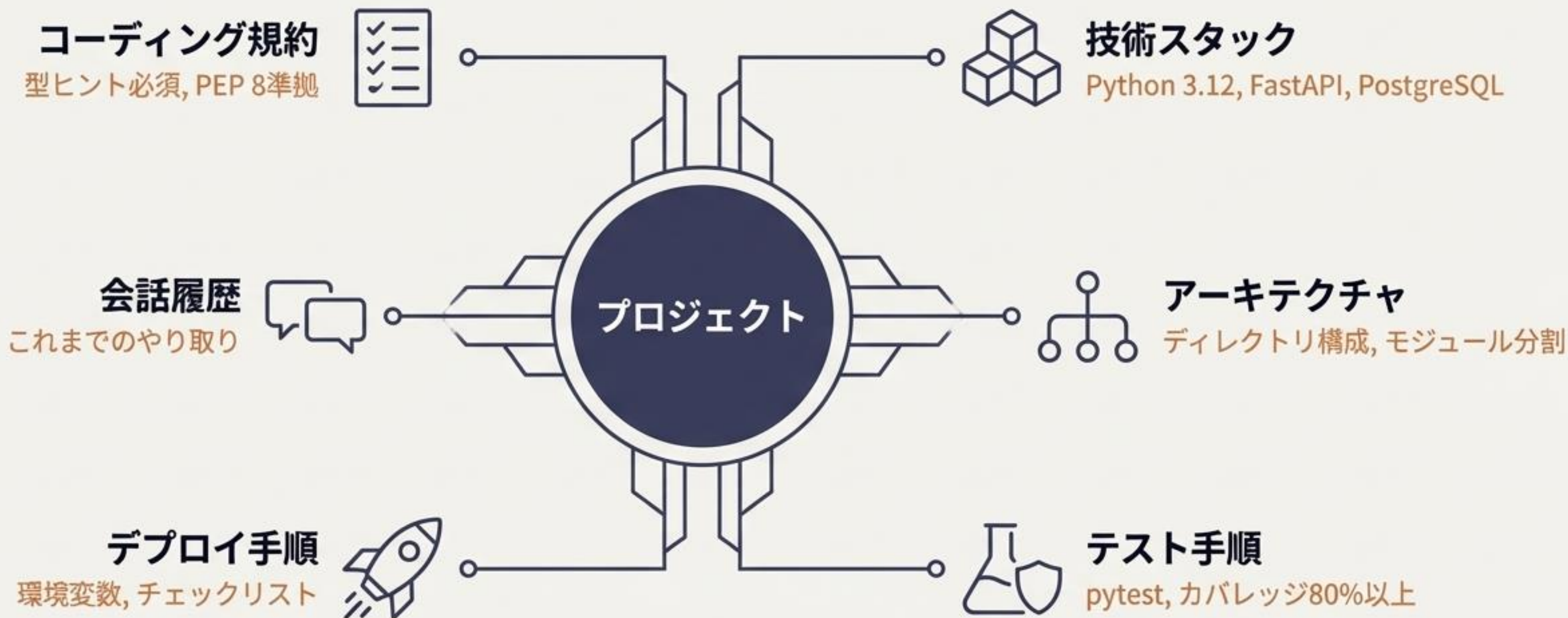
忘れられた情報

- 机が広い → たくさんの資料を広げて作業できる。
- 机が狭い → 資料を置ききれず、一部は床に落ちる（AIは参照できなくなる）。

重要な帰結: 会話が長くなりコンテキストが限界を超えると、古い情報から削除される。

プロジェクトのコンテキストは、想像以上に膨大である

適切なコードを書くためには、**これら全てを踏まえる**必要がある。



プロンプトエンジニアリングの限界

旧来のアプローチとその問題点

1. 毎回コピペする
→ 非効率。
2. システムプロンプトに全て書き込む
→ コンテキストの肥大化。

コストの増大



不要な情報も常に送信され、
トークンを消費する。

品質の低下



重要な指示がノイズに埋もれ、
見落とされる。

ハルシネーションの誘発



過剰な情報が、事実でない情
報の生成を誘発する。

レスポンス速度の低下



コンテキストが大きいほど
処理時間が長くなる。

新たなパラダイム：コンテキストエンジニアリング

定義： AIに渡す情報（コンテキスト）を管理する技術。
何を、いつ、どの形式で渡すかを最適化する。

1

分割
(Partitioning)



情報を小さく分ける。

2

局所化
(Localization)



必要な文脈だけを適用する。

3

段階的開示
(Progressive Disclosure)



必要になったタイミングで
初めて読む。

ゴール: 必要な情報だけを、必要なタイミングで渡す。

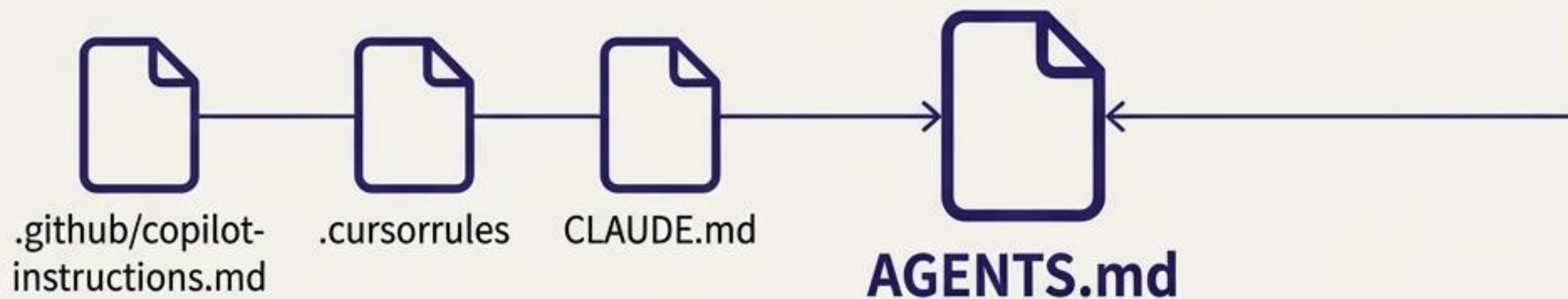
Solution 1: コンテキストファイル - 「AIのためのREADME」

コンセプト: プロジェクト固有のルールや規約を記載するファイル。チャットセッション開始時に自動でシステムプロンプトに読み込まれる。

利点: 毎回コピペする必要がなくなる。

主な記載内容:

-  技術スタック（使用言語、フレームワーク）
-  基本方針（コーディング規約）
-  禁止事項（やってはいけないこと）



AGENTS.md として標準化が進んでいる。

Problem: コンテキストファイルの肥大化 → Solution: 分割

課題: コンテキストファイルが大きくなると、結局システムプロンプトに全てを詰め込むのと同じ問題（トークン効率の低下、管理の困難化）が発生する。

1. 階層による制御

仕組み: ディレクトリ構造を利用。作業中の場所に最も近い `AGENTS.md` が優先的に読み込まれる。



2. 条件による制御

仕組み: YAMLフロントマターで適用対象ファイルをパターンマッチングで指定。

```
---
applyTo: "tests/**/*.py"
---
# テストファイル用ルール
```



Solution: オンデマンドなプロンプト - スラッシュコマンド

コンテキストファイル

常駐 (Persistent)

チャット開始時に読み込まれる

スラッシュコマンド

オンデマンド (On-Demand)

コマンド実行時に読み込まれる

ユースケース

/test

テスト実行の
手順を読み込む

/review

コードレビューの
観点を読み込む

/deploy

デプロイ手順を
読み込む

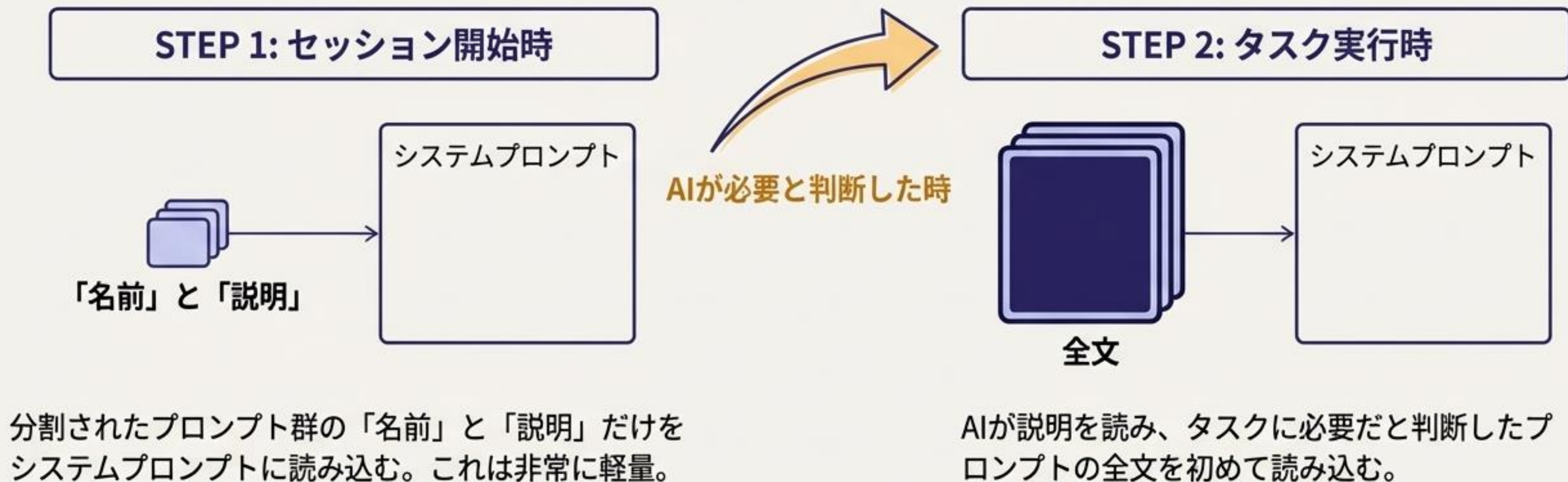
実装例 (ディレクトリ)

- .github/prompts/*.prompt.md
- .claude/commands/*.md
- .cursor/commands/*.md

必要な情報だけを、必要なタイミングで渡す。

統一原則：段階的開示（Progressive Disclosure）

必要な情報だけを、必要なタイミングで読み込むための最も効率的な仕組み。



適用先

カスタムエージェント

エージェントスキル

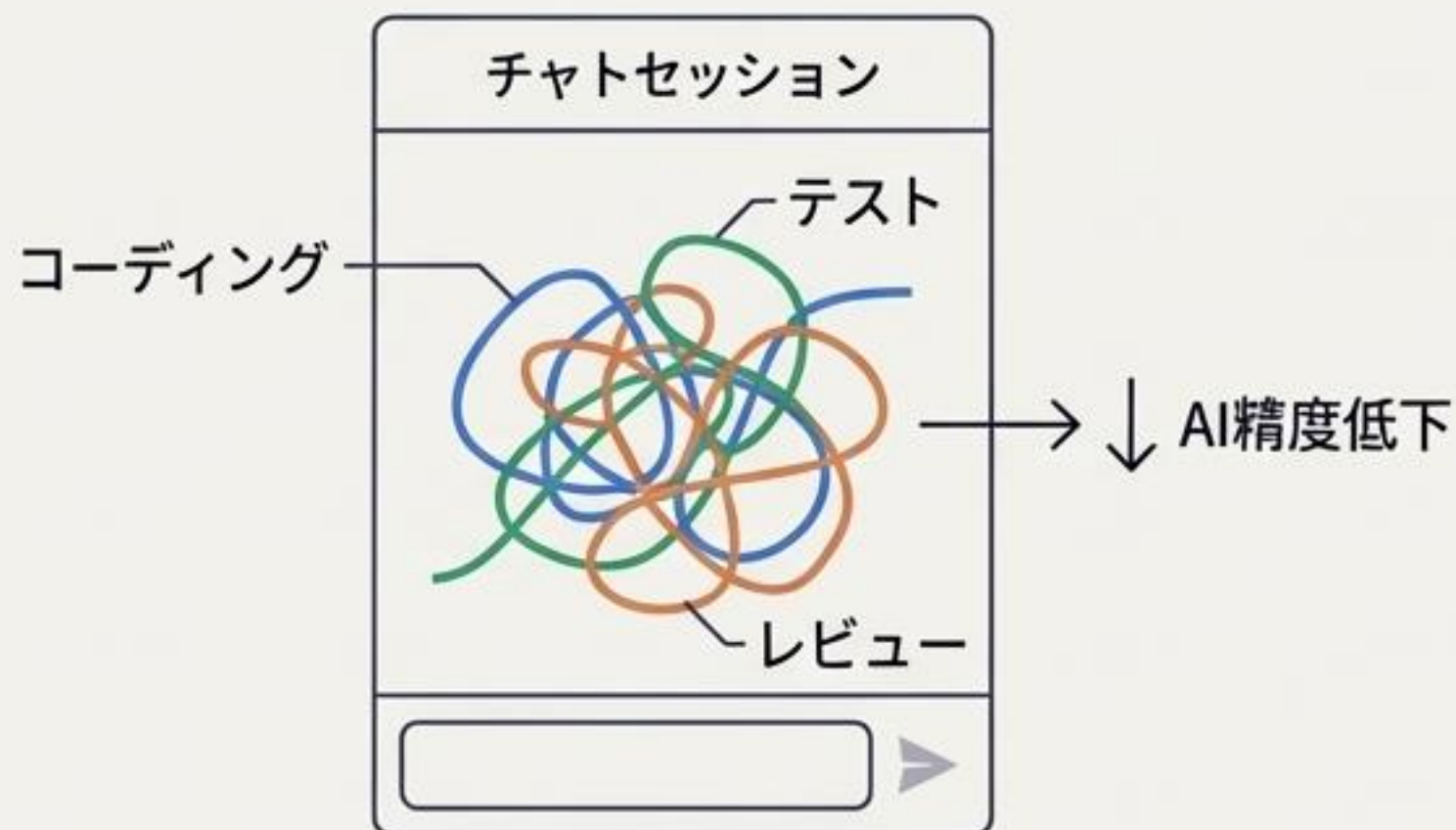
Problem: コンテキストの混濁 → Solution: カスタムエージェント

課題

一つのセッションで多様なタスク（コーディング、テスト、レビュー）を行うと、関連性のないコンテキストが混ざり、AIの精度が低下する。

解決策

役割ごとに専門化されたAIエージェントを作成する。各エージェントは特定のタスクに集中し、コンテキストの純度が向上する。



テスト専門AI

セキュリティレ
ビュー専門AI

デプロイ専門AI



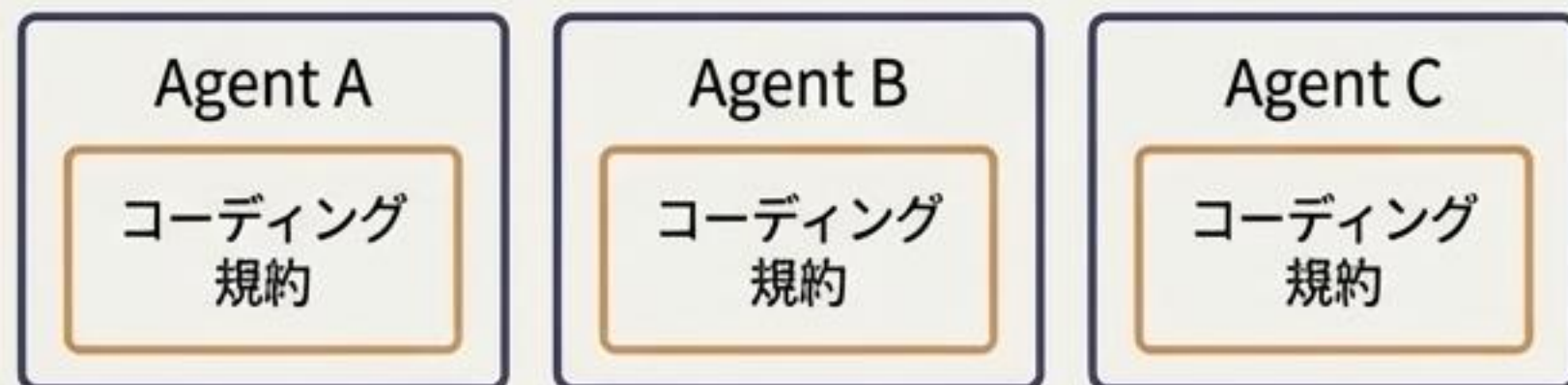
段階的開示の活用

セッション開始時には各エージェントの「名前」と「説明」だけを読み込み、AIが必要な専門家をタスクに応じて呼び出す。

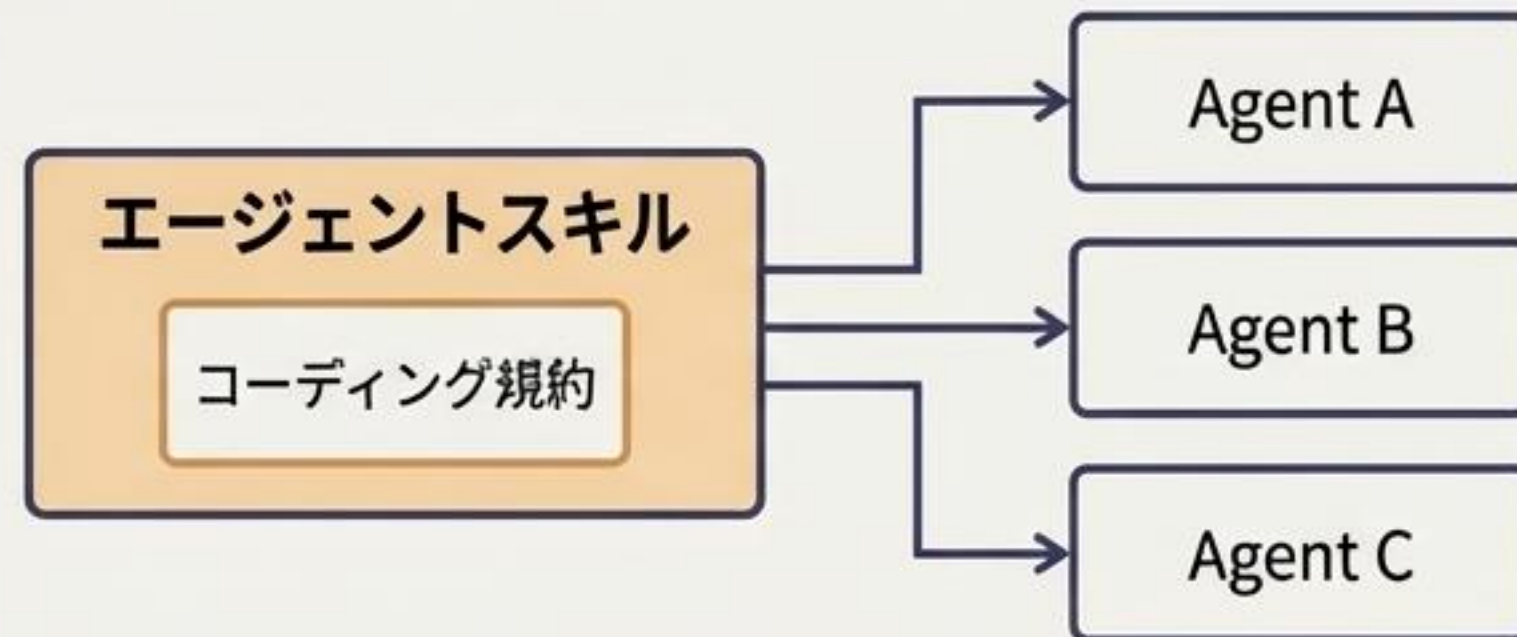
実装例: GitHub Copilotでは`.github/agents/\*.agent.md`で定義。

Problem: 知識の重複 → Solution: エージェントスキル

Before (課題: 知識の重複)



After (解決策: エージェントスキル標準)

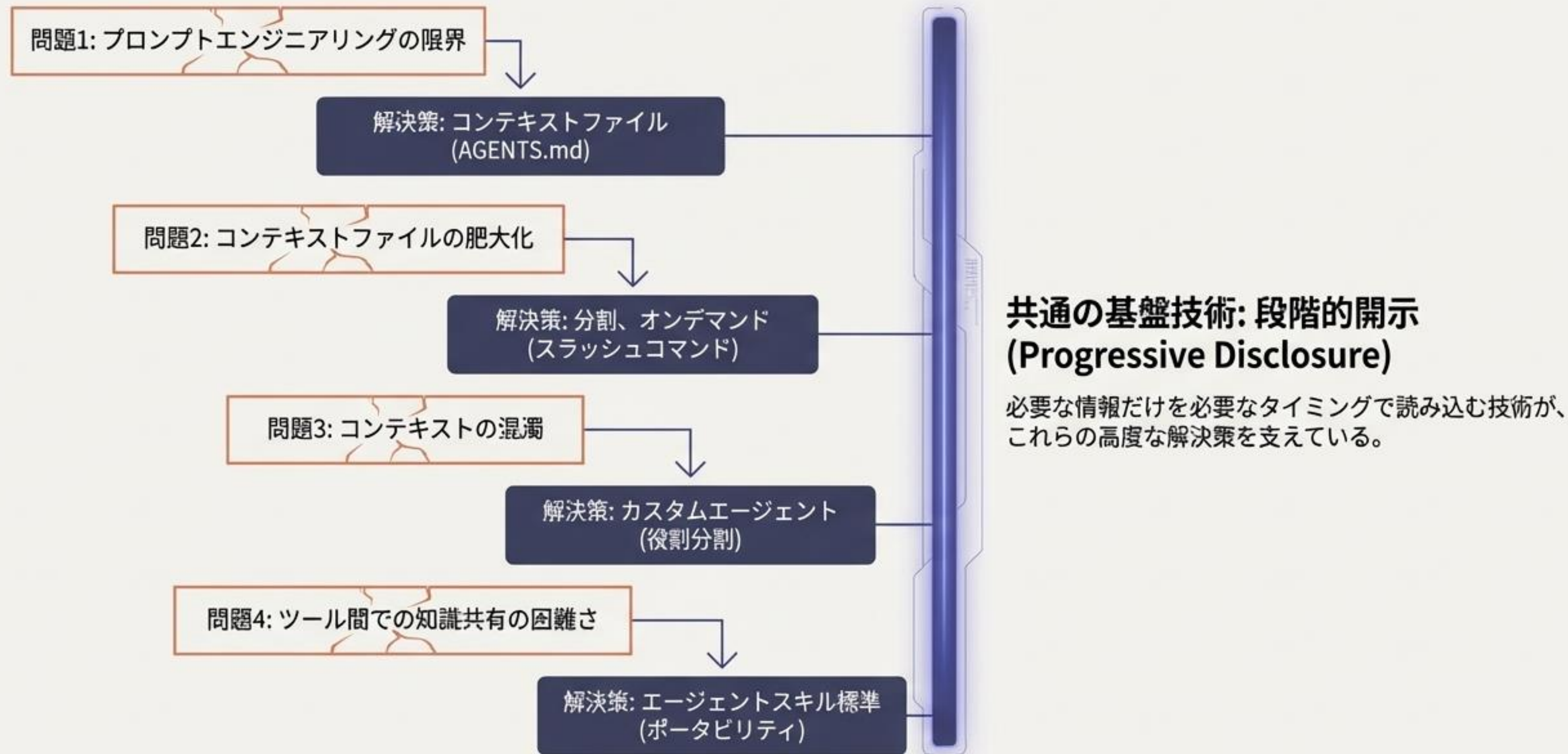


コンセプト エージェント = 「ペルソナ」 + 「スキル」 スキルを独立させ、コンポーネント化する。
in Noto Sans JP Regular

結果  スキルは再利用可能になり、複数のAIツール間で共有できる (ポータビリティ)。
 Single Source of Truthが実現する。

実装例 ``.\claude/skills/`` や ``.\github/skills/`` ディレクトリにスキルを配置。
各スキルは必須の ``SKILL.md`` (名前と説明を記載) を持つ。

コンテキストエンジニアリングの発展：問題と解決の連鎖



これは単なる技術ではない。業界標準である。

主要な組織と標準

Agentic AI Foundation (AAIF)

- 設立: Linux Foundationが2025年12月に設立。
- 目的: AGENTS.md, MCP, gooseなどの技術プロジェクトを管理。

エージェントスキル標準

- 提唱: Anthropic
- 公式サイト: agentskills.io

エコシステムの広がり

Adoption Stat: `AGENTS.md` は既に60,000以上のプロジェクトで採用。



ANTHROPIC



AAIF



Block

Bloomberg



AAIFのプラチナメンバーには、業界の主要企業が名を連ねる。